Lógica para Engenharia de Software

Introdução à Análise de Algoritmos

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



O que é um Algoritmo?

Definição Informal

Um **algoritmo** é qualquer procedimento computacional bem definido que toma algum valor (ou conjunto de valores) como **entrada** e produz algum valor (ou conjunto de valores) como **saída**.

- É uma sequência de etapas computacionais que transformam a entrada na saída.
- Pode ser visto como uma ferramenta para resolver um problema computacional bem especificado.



O Algoritmo como Ferramenta

- O **enunciado do problema** especifica a relação desejada entre entrada e saída.
- O **algoritmo** descreve o procedimento específico para alcançar essa relação.

Exemplo: Ordenação

Necessidade de ordenar uma sequência de números em ordem não decrescente.

- **Entrada**: Uma sequência de n números $\langle a_1, a_2, \ldots, a_n \rangle$.
- **Saída**: Uma reordenação $\langle a'_1, a'_2, \ldots, a'_n \rangle$ tal que $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

Instâncias e Problemas

Instância de um Problema

Consiste na entrada necessária para calcular uma solução para o problema (desde que satisfaça as restrições impostas).

Exemplo de Instância de Ordenação:

- **Entrada**: $\langle 31, 41, 59, 26, 41, 58 \rangle$
- **Saída**: $\langle 26, 31, 41, 41, 58, 59 \rangle$

A ordenação é fundamental: muitos programas a utilizam como etapa intermediária.

Fatores na Escolha de um Algoritmo

Não existe um "melhor"algoritmo universal. A escolha depende de:

- Número de itens a ordenar.
- Grau de ordenação inicial.
- Restrições aos valores.
- Arquitetura do computador.
- Tipo de dispositivo de armazenamento (RAM, disco, fita).

Análise de Algoritmos

Nos ajuda a escolher a ferramenta certa para cada solo fértil de projeto.

Algoritmo Correto

Diz-se que um algoritmo é **correto** se, para toda instância de entrada, ele parar com a saída correta.

- **Algoritmo Incorreto**: Poderia não parar em algumas instâncias ou parar com uma resposta errada.
- **Utilidade**: Algoritmos incorretos podem ser úteis se pudermos controlar sua *taxa de erro* (ex: teste de primalidade para grandes números).

A especificação pode ser em linguagem comum, código de computador ou projeto de hardware.

Aplicações Práticas (I)

Os algoritmos estão por toda parte:

- **Projeto Genoma Humano**: Identificar 100.000 genes no DNA humano e analisar 3 bilhões de pares de bases químicas.
- **Internet**: Gerenciamento de grandes volumes de dados, determinação de boas rotas de transmissão e mecanismos de busca rápida.
- **Comércio Eletrônico**: Criptografia de chave pública e assinaturas digitais para garantir o sigilo de senhas e cartões.

Aplicações Práticas (II)

- **Alocação de Recursos**: Programação linear para maximizar lucros ou minimizar custos (ex: localização de poços de petróleo, escalas de tripulação aérea).
- **Mapas e GPS**: Determinar a rota mais curta entre duas interseções em um grafo rodoviário.
- **Similaridade de DNA**: Determinar a *subsequência comum mais longa* entre duas sequências genéticas.

Economia de tempo (humano e máquina) e dinheiro!

Características de Problemas Algorítmicos

A maioria dos problemas interessantes compartilha duas características:

1. **Escala de Soluções**: Possuem muitas soluções candidatas, mas a grande maioria não resolve o problema. Encontrar a "melhor"é o desafio.
2. **Utilidade Prática**: Resolvem questões reais de eficiência, segurança e logística.

Exemplos Avançados

Ordenação Topológica, Envoltória Convexa, Transformada Rápida de Fourier (FFT).

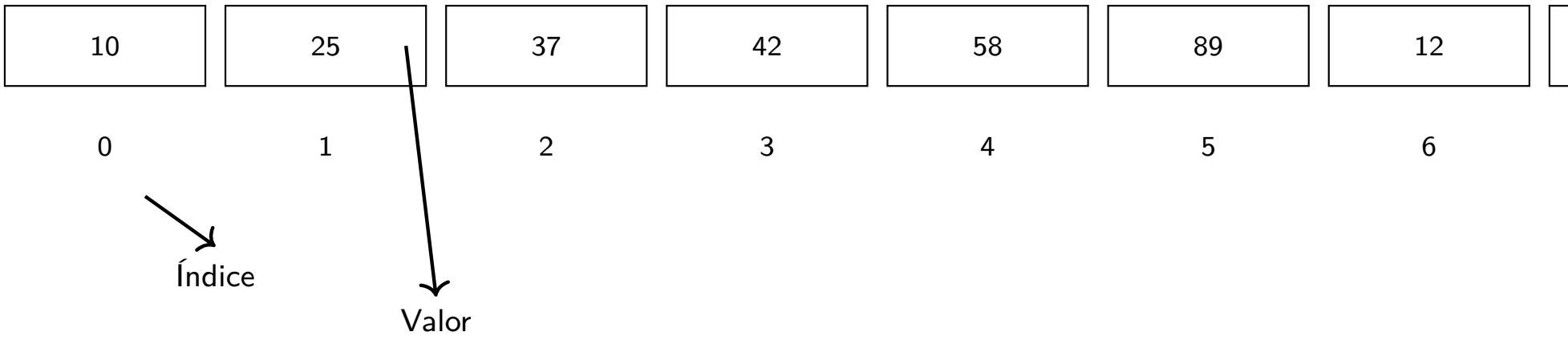
O que é uma Estrutura de Dados?

Definição

Uma **estrutura de dados** é um modo de armazenar e organizar dados com o objetivo de facilitar acesso e modificações.

- Nenhuma estrutura única funciona bem para todas as finalidades
- Conhecer pontos fortes e limitações é essencial
- A escolha afeta diretamente a eficiência dos algoritmos

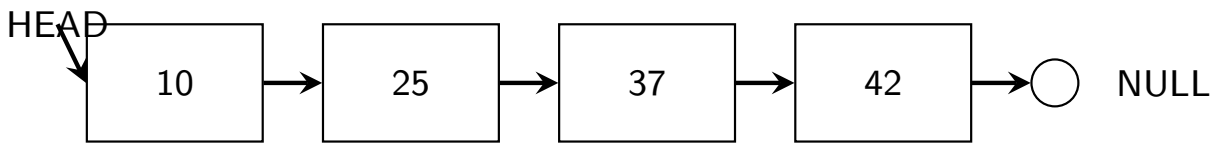
Estruturas de Dados: Array (Vetor)



Características

- Acesso direto por índice: $O(1)$
- Inserção/Deleção: $O(n)$
- Tamanho fixo
- Memória contínua

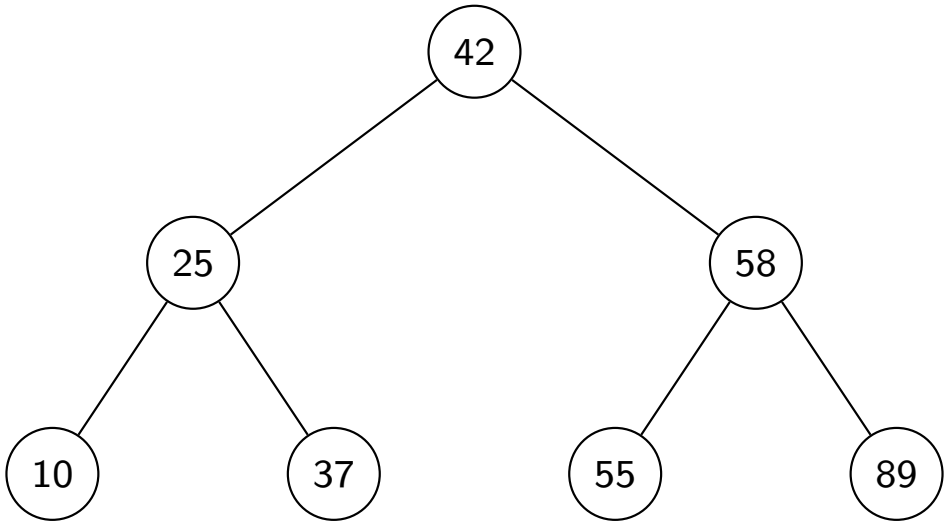
Estruturas de Dados: Lista Encadeada



Características

- Acesso sequencial: $O(n)$
- Inserção/Deleção: $O(1)$ no início/meio
- Tamanho dinâmico
- Memória não contínua

Estruturas de Dados: Árvore Binária



Características

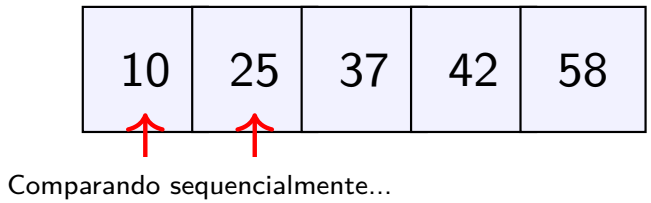
- Busca em árvores balanceadas: $O(\log n)$
- Inserção/Deleção: $O(\log n)$
- Estrutura hierárquica
- Uso em bancos de dados, expressões, Huffman

Busca em Vetor (Busca Linear)

A **busca linear** (ou sequencial) é o método mais simples de encontrar um elemento em uma estrutura de dados linear.

Pseudocódigo

para $i \leftarrow 0$ **até** $n - 1$ **faça**
    se $vetor[i] == alvo$ **então**
        retornar i *(Sucesso)*
fim para
retornar -1 *(Não encontrado)*



Comparando sequencialmente...

Melhor e Pior Caso na Busca em Vetor

A eficiência da busca linear depende de onde o elemento está localizado.

Melhor Caso

O elemento buscado está na **primeira posição**.

- Apenas 1 comparação.
- Complexidade: $O(1)$.

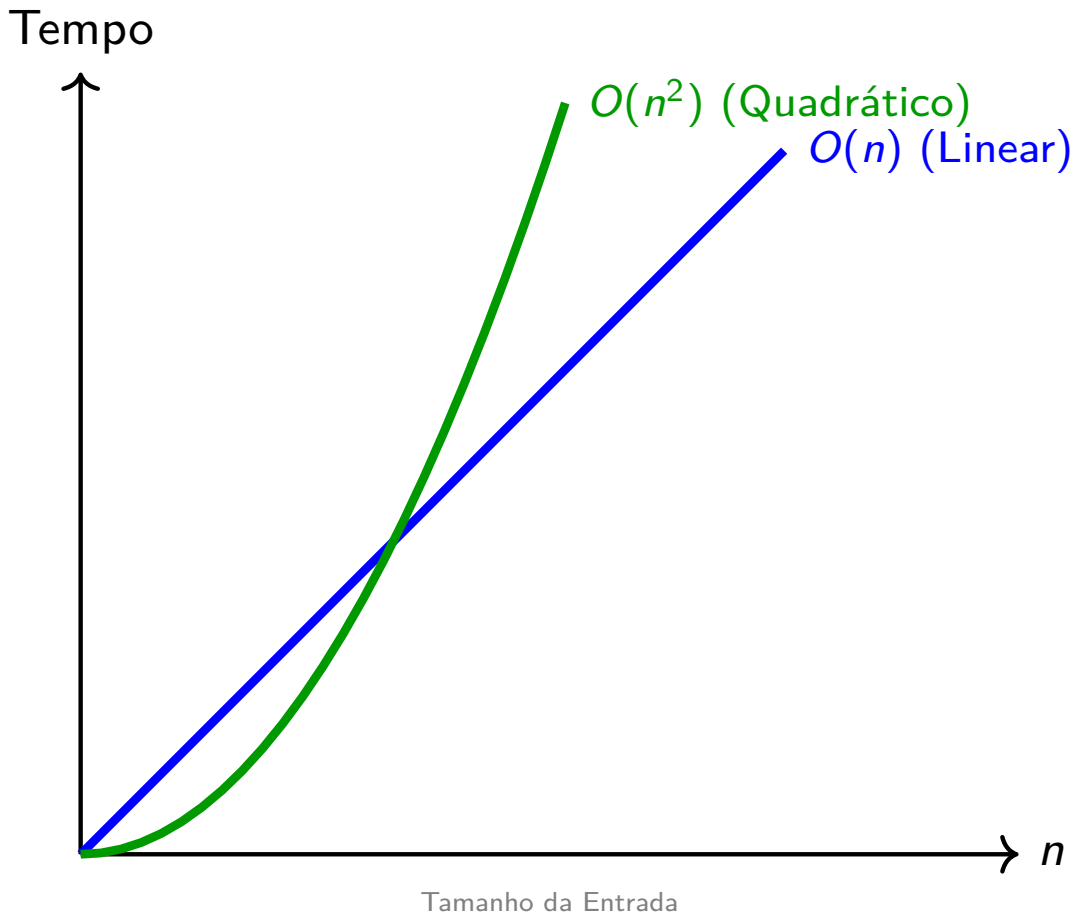
Pior Caso

O elemento está na **última posição** ou **não existe**.

- n comparações.
- Complexidade: $O(n)$.

Comparação: Crescimento Linear vs Quadrático

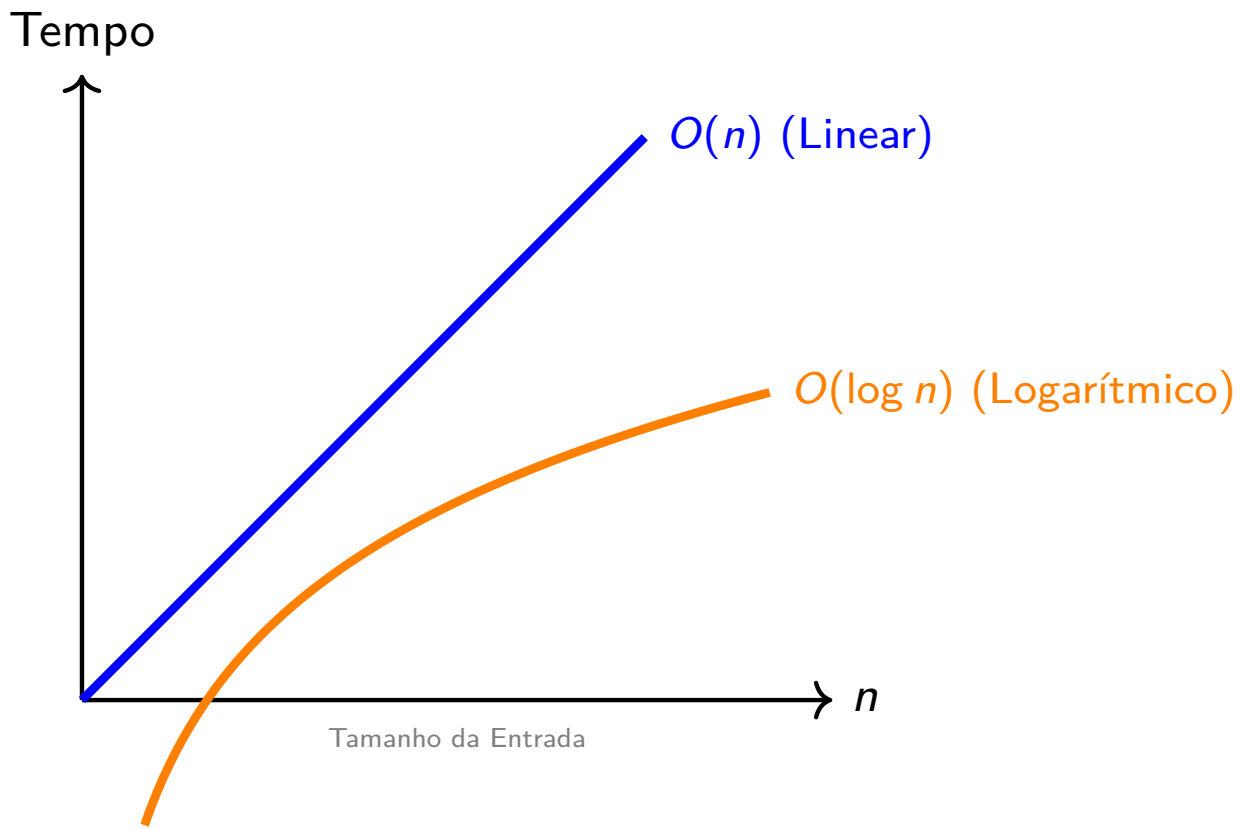
Visualização do impacto da complexidade no tempo de execução à medida que o tamanho da entrada (n) cresce.



- **Linear**: O tempo cresce na mesma proporção que a entrada.
- **Quadrático**: Se a entrada dobra ($2n$), o tempo **quadruplica** ($4\times$).

Comparação: Crescimento Logarítmico vs Linear

Visualização da alta eficiência dos algoritmos logarítmicos em comparação aos lineares.



- **Linear**: O esforço cresce diretamente com o número de elementos.
- **Logarítmico**: O esforço cresce de forma mínima, mesmo com entradas massivas (ex: Busca Binária).

O que é o Pior Caso de um Algoritmo?

Definição

O **Pior Caso** representa o tempo máximo de execução de um algoritmo para qualquer entrada de tamanho n.

- É a **garantia** de que o algoritmo nunca demorará mais do que aquele tempo.
- É a métrica mais utilizada na prática, pois permite planejar o sistema para situações críticas.
- Representado pela notação O (Big-O) como um limite superior.

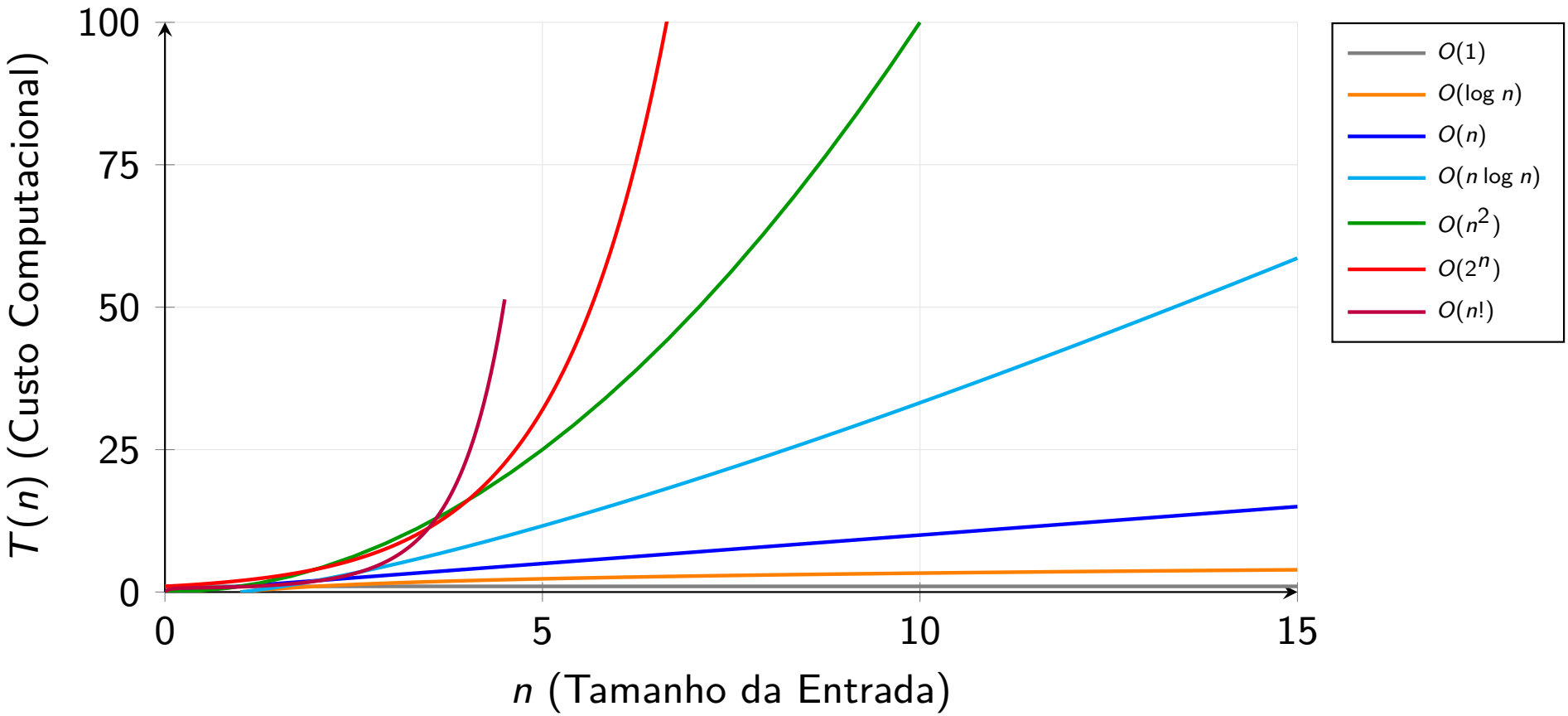
Exemplos de Pior Caso (O)

Como diferentes algoritmos se comportam no seu cenário mais difícil:

Algoritmo	**Pior Caso**	**Cenário**
Busca Linear	$O(n)$	Elemento no final ou ausente
Busca Binária	$O(\log n)$	Elemento no início/fim das divisões
Bubble Sort	$O(n^2)$	Vetor em ordem decrescente
Quick Sort	$O(n^2)$	Pivô sempre escolhe o maior/menor

Gráfico de Comparação de Complexidades (Big-O)

Resumo visual de como as funções de custo se comportam à medida que a entrada cresce:



O que é o Melhor Caso de um Algoritmo?

Definição

O **Melhor Caso** representa o tempo mínimo de execução de um algoritmo para uma entrada de tamanho n.

- Ocorre quando a entrada é a mais favorável possível para a lógica do algoritmo.
- É útil para mostrar o potencial de algoritmos otimizados que "desistem"cedo quando o trabalho está feito.
- Representado pela notação Ω (Ômega) como um limite inferior.

Exemplos de Melhor Caso (Ω)

O comportamento quando a entrada é ideal:

Algoritmo	**Melhor Caso**	**Cenário**
Busca Linear	$\Omega(1)$	Elemento na 1ª posição
Busca Binária	$\Omega(1)$	Elemento no meio exato (1ª tentativa)
Insertion Sort	$\Omega(n)$	Vetor já ordenado
Merge Sort	$\Omega(n \log n)$	Sempre divide e conquista

Pseudocódigo da Busca Linear

Apresentação do algoritmo no formato clássico para análise:

	1	2	3	4	5
A:	31	41	59	26	41

BUSCA-LINEAR(A, v)

	instrução
1	**for** $i = 1$ **to** $A.length$
2	**if** $A[i] == v$
3	**return** i
4	**return** NIL

Cálculo do Custo Computacional

	1	2	3	4	5
A:	31	41	59	26	41

Para analisar um algoritmo, atribuímos um custo c_i a cada linha:

BUSCA-LINEAR(A, v)

	instrução	custo	vezes
1	**for** $i = 1$ **to** $A.length$	c_1	$n + 1$
2	**if** $A[i] == v$	c_2	n
3	**return** i	c_3	0 ou 1
4	**return** NIL	c_4	1

O tempo total $T(n)$ é a soma dos produtos (Custo $\times$ Vezes):

$$T(n) = c_1(n+1) + c_2(n) + c_3(0 \text{ ou } 1) + c_4(1)$$

A Função de Tempo $T(n)$

No cenário onde o elemento **não é encontrado** (Pior Caso):

- A linha 1 executa $n + 1$ vezes.
- A linha 2 executa n vezes.
- A linha 3 nunca executa (0).
- A linha 4 executa 1 vez.

Equação do Tempo

$$T(n) = c_1(n + 1) + c_2(n) + c_4$$
$$T(n) = (c_1 + c_2)n + (c_1 + c_4)$$

Observe que $T(n)$ é uma **função linear** da forma $f(n) = an + b$.

Análise do Pior Caso

Ao analisar o custo em larga escala, focamos no termo de **maior crescimento**:

- Em $T(n) = (c_1 + c_2)n + (c_1 + c_4)$, o termo dominante é n.
- As constantes (c_1, c_2, c_4) dependem do hardware e compilador, por isso as ignoramos.

Conclusão

Dizemos que a complexidade de tempo no pior caso para a busca linear é **proporcional a** n.

$$T(n) \in O(n)$$

Pseudocódigo: Insertion Sort

O **Insertion Sort** (Ordenação por Inserção) é um exemplo clássico de algoritmo com custo quadrático.

	1	2	3	4	5	6
A:	5	2	4	6	1	3

INSERTION-SORT(A)

	instrução
1	**for** $j = 2$ **to** $A.length$
2	$key = A[j]$
3	// *Insere* $A[j]$ *na sequência ordenada* $A[1 \ldots j-1]$
4	$i = j - 1$
5	**while** $i > 0$ **and** $A[i] > key$
6	$A[i+1] = A[i]$
7	$i = i - 1$
8	$A[i+1] = key$

Análise de Custo (Pior Caso)

No pior caso (vetor em ordem decrescente), o laço **while** executa o máximo de vezes.

	instrução	custo	vezes
1	**for** $j = 2$ **to** n	c_1	n
2	$key = A[j]$	c_2	$n-1$
4	$i = j - 1$	c_4	$n-1$
5	**while** $i > 0$ **and** $A[i] > key$	c_5	$\sum_{j=2}^{n} j$
6	$A[i+1] = A[i]$	c_6	$\sum_{j=2}^{n} (j-1)$
8	$A[i+1] = key$	c_8	$n-1$

No pior caso, temos: $\sum_{j=2}^{n} j = \frac{n^2+n}{2} - 1$ e $\sum_{j=2}^{n} (j-1) = \frac{n^2-n}{2}$.

Cálculo de $T(n)$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n^2+n}{2} - 1\right) + c_6\left(\frac{n^2-n}{2}\right) + c_8(n-1)$$

$$T(n) = \left(\frac{c_5+c_6}{2}\right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5-c_6}{2} + c_8\right) n - (c_2 + c_4 + c_5 + c_8)$$

Conclusão: Complexidade Quadrática

Ao somar todos os custos, obtemos uma função polinomial de grau 2:

Função de Tempo

$$T(n) = an^2 + bn + c$$

- O termo dominante é n^2.
- Para valores grandes de n, os termos n e constantes tornam-se insignificantes.

Complexidade Final

Dizemos que o *Insertion Sort* possui complexidade de pior caso:

$$T(n) \in O(n^2)$$

Pseudocódigo: Busca Binária

A **Busca Binária** é o exemplo fundamental de eficiência logarítmica, aplicável em vetores **ordenados**.

	1	2	3	4	5	6	7
A:	10	25	37	42	55	58	89

BUSCA-BINARIA(A, v, p, r)

	instrução
1	**while** $p \leq r$
2	$q = \lfloor (p + r)/2 \rfloor$
3	**if** $A[q] == v$ **return** q
4	**else if** $A[q] < v$
5	$p = q + 1$
6	**else** $r = q - 1$
7	**return** NIL

Análise de Custo (Logarítmica)

Diferente da busca linear, a cada iteração o "espaço de busca"é reduzido pela **metade**.

- **Iteração 0**: tamanho do vetor é n.
- **Iteração 1**: tamanho do vetor é $n/2$.
- **Iteração 2**: tamanho do vetor é $n/4$ (ou $n/2^2$).
- **Iteração** k: tamanho do vetor é $n/2^k$.

Quando o algoritmo para?

No pior caso, a busca continua até que o espaço de busca seja 1:

$$\frac{n}{2^k} = 1 \implies 2^k = n \implies k = \log_2 n$$

Conclusão: Complexidade Logarítmica

O número de comparações cresce de forma extremamente lenta em relação ao tamanho da entrada n.

Tamanho n	**Comparações ($\approx \log_2 n$)**
10	4
1.000	10
1.000.000	20
1.000.000.000	30

Complexidade de Pior Caso

Dizemos que a busca binária possui complexidade:

$$T(n) \in O(\log n)$$

Pseudocódigo: Fibonacci Recursivo

O cálculo recursivo da sequência de Fibonacci é um exemplo clássico de algoritmo com custo **exponencial**.

FIBONACCI(n)

	instrução
1	**if** $n \leq 1$
2	**return** n
3	**else**
4	**return** FIBONACCI$(n-1)$ + FIBONACCI$(n-2)$

Cálculo do Custo (Fibonacci)

Para algoritmos recursivos, o custo é expresso através de uma **Equação de Recorrência**.

Análise de Instruções

	instrução	custo	vezes
1	**if** $n \leq 1$	c_1	1
2	**return** n	c_2	0 ou 1
4	**return** FIB$(n-1)$ + FIB$(n-2)$	c_3	$T(n-1) + T(n-2)$

Para analisar o crescimento, usamos a substituição simplificada:

Expansão da Recorrência

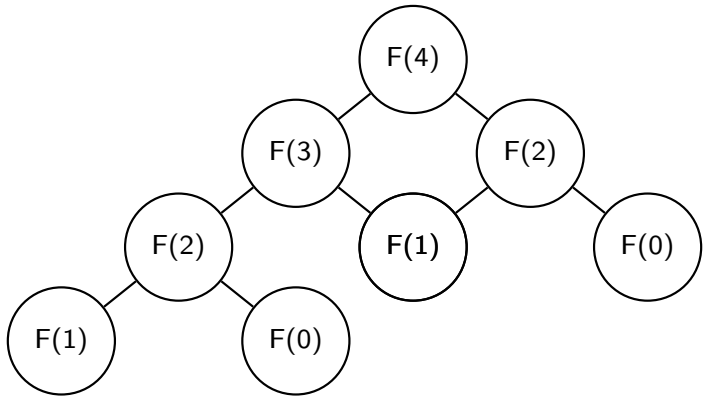
Como $T(n-1) > T(n-2)$, podemos dizer que $T(n) \approx 2 \cdot T(n-1)$:

- $T(n) = 2 \cdot T(n-1) + c$
- $T(n) = 2 \cdot (2 \cdot T(n-2) + c) + c = 2^2 T(n-2) + 3c$
- $T(n) = 2^3 T(n-3) + 7c$
- ...
- $T(n) = 2^n T(0) + (2^n - 1)c$

Isso prova que o custo total é **proporcional a** 2^n.

Análise de Custo (Exponencial)

A cada chamada da função, o algoritmo gera outras **duas chamadas** recursivas, criando uma árvore de execução.



- A árvore de chamadas cresce em largura à medida que aumenta a profundidade n.
- O número total de operações dobra aproximadamente a cada incremento em n.

Conclusão: Complexidade Exponencial

Algoritmos exponenciais são considerados **inviáveis** para entradas de tamanho médio ou grande.

Tamanho n	**Operações ($\approx 2^n$)**
10	1.024
20	$\approx$ 1 milhão
40	$\approx$ 1 trilhão
100	Mais que átomos no universo

Complexidade de Pior Caso

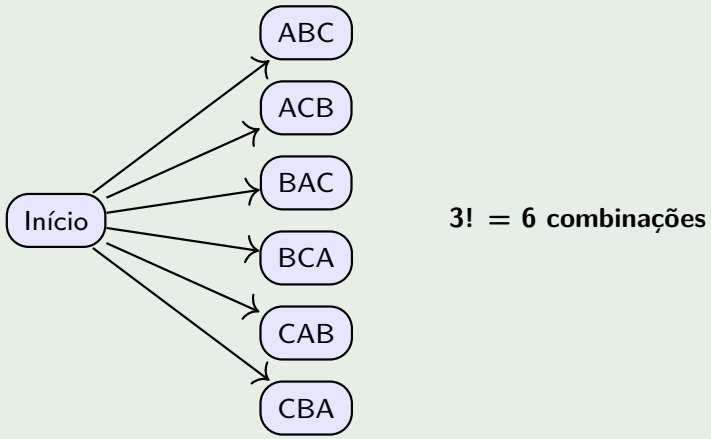
Dizemos que o Fibonacci recursivo possui complexidade:

$$T(n) \in O(2^n)$$

Segurança: Teste de Força Bruta (Brute Force)

Em segurança de software, um ataque de **força bruta** tenta todas as combinações possíveis para encontrar uma chave ou senha secreta.

Exemplo: Senha com $\{A, B, C\}$ ($n = 3$)



3! = 6 combinações

QUEBRA-SENHA(S, n)

```
1   para cada permutação P de S:
2     se TentaLogin(P) == SUCESSO:
3       return P
```

Cálculo do Custo Computacional

Para analisar o custo, atribuímos c_i a cada linha e contamos as execuções no **pior caso**.

	instrução	**custo**	**vezes**
1	**para cada** permutação P	c_1	$n!$
2	**se** TentaLogin(P)	c_2	$n!$
3	**return** P	c_3	1

Custo Total $T(n)$

No pior caso (senha é a última permutação testada):

$$T(n) = c_1(n!) + c_2(n!) + c_3(1) \implies O(n!)$$

Análise do Pior Caso ($O(n!)$)

O crescimento fatorial é o mais agressivo de todos. Pequenos aumentos em n tornam o sistema **impenetrável**.

- **Cenário** $n = 10$: 3.628.800 tentativas. (Segundos)
- **Cenário** $n = 13$: 6.227.020.800 tentativas. (Horas)
- **Cenário** $n = 20$: $2,4 \times 10^{18}$ tentativas. (**Séculos**)

Conclusão de Engenharia

Quanto maior o número de caracteres (n) em uma senha sem repetição, maior a barreira contra ataques de força bruta, devido à natureza do $O(n!)$.

A segurança moderna depende da inviabilidade computacional de testar todas as chaves.